

EE4410 CYBER SECURITY OF POWER GRIDS

Assignment 2: Cryptography Applications

May 28, 2025

Zubair Al-Zubi (5342007)

[Link to Colab Notebook](#)

This lab report covers the topic of Cryptography Applications and discusses experiments done with symmetric encryption (AES), asymmetric encryption (RSA), and digital signatures (RSASSA-PSS and ECDSA). The primary objective is to understand secure communication principles and compare the behavior, strengths, and limitations of each cryptographic scheme.

1 Symmetric Encryption

In the first part of the lab, AES encryption was implemented using various modes of operation and key lengths. The primary goal was to securely exchange messages between two parties via an untrusted medium (in this case, the public API service JSONbin.io).

The encryption process began by converting a plaintext message into ciphertext using a pre-shared secret password. The AES key was derived from this password, and the encryption was performed using the AES-GCM mode with a 256-bit key. The encrypted message was then uploaded to a publicly accessible bin on JSONbin.io. On the receiving end, the recipient retrieved the ciphertext and successfully decrypted it using the same secret, AES mode, and key length. This encryption/decryption process is illustrated below:

Plaintext: Hello! This is my super secret message

Ciphertext: YLnGtP9D0py0/R30tBY3/zxtsvvAn/LLP0jdX4dNb7uu9Yzsk+M/
erAJwC7+XZ+CcCpGVZBSJMTcgei5ubcM4GLA

Uploaded to my bin with ID: 6836102f8561e97a501c4973

Retrieved friend's payload: {'ciphertext':
'lNyY01dUqwty8yBWhDsgnY3c3KCITMA5Vrru5tX7Md9++iHe0VIY', 'mode': 'AES.MODE_GCM',
'key_length': 256}

Decrypted friend's message: hello world

To enable this exchange, both parties needed to share the same secret passphrase, the AES mode used (e.g., GCM, ECB, etc.), and the key length (128, 192, or 256 bits).

Figure 1 shows the uploaded payload to JSONbin.io.

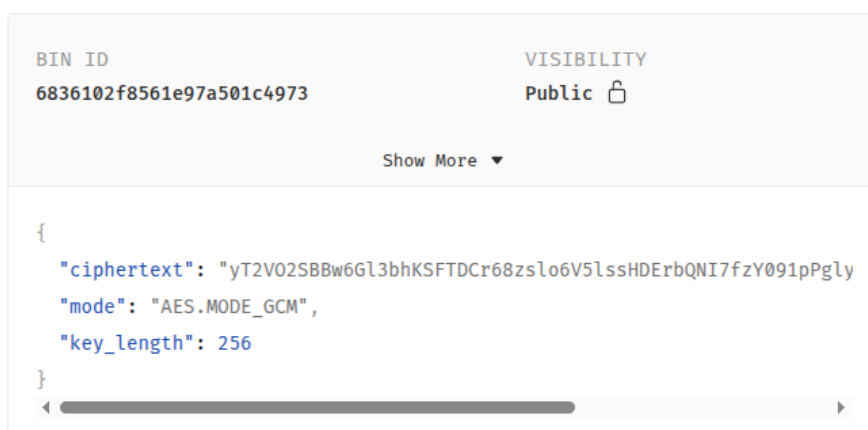


Figure 1: The ciphertext uploaded to JSONbin.io

1.1 Discussion

Different AES modes and key lengths affect the ciphertext and its security. For example, ECB mode, although simple, is insecure because it reveals patterns. CBC, CTR, and CFB modes require an IV (16 bytes), which increases security. AEAD modes like GCM include a nonce and an authentication tag, providing data integrity and increasing ciphertext size. Longer keys significantly improve security by making brute-force attacks more difficult [1], [2], [3].

In summary, the AES mode determines how secure and large the ciphertext will be, and the key length defines how computationally expensive it is to break the encryption. AEAD modes are generally preferred for their integrity guarantees, especially when messages are sent over untrusted channels like JSONbin.

2 Asymmetric Encryption

In the second part of the lab, asymmetric encryption was carried out using the RSA public-key system. The primary goal was to securely exchange a confidential message without relying on a pre-shared secret. Unlike symmetric encryption, RSA enables secure key exchange over untrusted networks by allowing encryption with a public key and decryption with the corresponding private key.

The experiment began with generating a new RSA key pair consisting of a private and a public key. Using the provided utility functions, a 2048-bit key pair was created. Both keys were then printed to verify correct generation. The private key remained confidential on the sender's device, while the public key was made available for use in encryption.

After that, the secure message exchange was performed. The recipient shared their public untampered RSA key, as it is the only required information. It was then imported using the provided `read_rsa_key()` function. A plaintext message was then encrypted using the recipient's public key via `encrypt_rsa()` and uploaded to JSONbin. The recipient downloaded the ciphertext from the shared bin and decrypted it using their own private key with the `decrypt_rsa()` function. The decrypted message was successfully recovered, as shown in Figure 2.

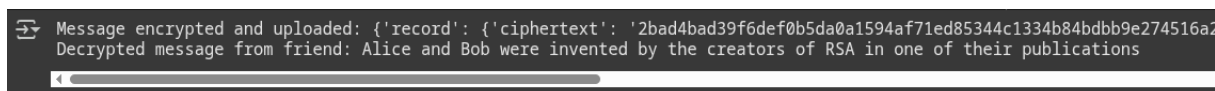


Figure 2: RSA message successfully decrypted and recovered

To understand the implications of key size, RSA keys of lengths 1024, 2048, 3072, and 4096 bits were generated and their internal parameters inspected. These included the modulus n , public exponent e , private exponent d , and the two large prime factors p and q . As expected, the modulus size directly corresponded to the key length, while d approached but remained slightly smaller than n in bit length. The primes p and q were each approximately half the size of n . The exponent e was consistently 65537, which is a common choice that balances security and performance.

2.1 Discussion

RSA encryption enables secure communication without requiring a pre-shared secret [4]. This is advantageous in situations where secure key distribution is impractical. However, RSA comes with trade-offs. Compared to symmetric encryption (like AES), RSA is computationally more expensive, especially for key generation and decryption. It also produces significantly longer ciphertexts.

Larger RSA key lengths provide higher security but at the cost of performance and larger message sizes. For instance, the 4096-bit key produced a much larger modulus and slower computations compared to a 1024-bit key. Nevertheless, 2048 bits is widely regarded as the current security baseline, while 3072 and 4096 bits are suitable for long-term confidentiality or sensitive data [5].

In summary, RSA supports secure message exchange by separating the roles of encryption (using the public key) and decryption (using the private key). This allows secure communication even over public channels like JSONbin, as long as key integrity and authenticity are maintained.

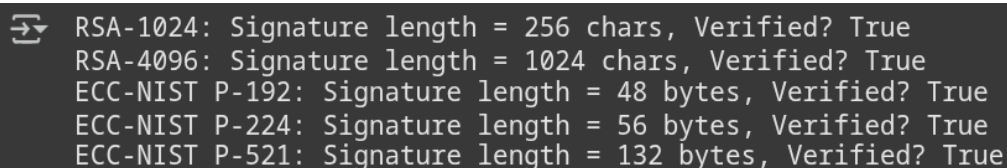
3 Digital Signatures

In the final part of the lab, digital signatures using two schemes: RSASSA-PSS (RSA) and ECDSA was performed. The aim was to guarantee both confidentiality and authenticity by encrypting a message and signing it using the sender's private key. The signature and the encrypted message were then uploaded to JSONbin.

The procedure followed was similar to the asymmetric encryption part. We first signed a plaintext message using the RSA private key with the RSASSA-PSS scheme. Next, the signed message was encrypted using the recipient's public RSA key. Both the ciphertext and the signature were packaged into a JSON object and published. The recipient then retrieved the message from JSONbin, verified the digital signature using the sender's public key, and finally decrypted the ciphertext using their private key.

The keys used for encryption and decryption were RSA public and private keys, respectively. Meanwhile, the signature was generated using the sender's private RSA key and verified using the corresponding public key.

In addition to RSA, we performed experiments with ECDSA using NIST curves (P-192, P-224, and P-521). We generated EC key pairs and signed the same message with each, then verified the signature using the EC public key. Across all curves, signature verification was successful, as shown in Figure 3.



```
➦ RSA-1024: Signature length = 256 chars, Verified? True
RSA-4096: Signature length = 1024 chars, Verified? True
ECC-NIST P-192: Signature length = 48 bytes, Verified? True
ECC-NIST P-224: Signature length = 56 bytes, Verified? True
ECC-NIST P-521: Signature length = 132 bytes, Verified? True
```

Figure 3: Generated signatures with different combination of (RSA, RSA_KEY_LENGTH) and (ECDSA, CURVES)

RSA key lengths of 1024 and 4096 bits produced signatures of 256 and 1024 characters, respectively. In contrast, ECDSA with NIST P-192, P-224, and P-521 yielded compact signatures of 48, 56, and 132 bytes, respectively. This confirms that ECC achieves the same security with significantly smaller key and signature sizes.

While RSA is more widely supported and simpler to implement, ECC provides stronger security-per-bit, shorter key lengths, and faster signature generation, which makes it suitable for constrained environments like IoT devices [6], [7].

3.1 Discussion

Digital signatures are essential for ensuring message authenticity. This lab demonstrated the full digital signature pipeline, including key generation, signing, verification, and hybrid encryption.

RSASSA-PSS and ECDSA both offer robust cryptographic guarantees. However, RSA keys and signatures are significantly larger than those in ECC. As shown earlier, a 4096-bit RSA key yields a 1024-character signature, whereas a 521-bit EC key generates a signature of only 132 bytes. The performance and bandwidth benefits of ECC make it a better choice in modern cryptographic systems, especially for resource-constrained platforms [7].

Bibliography

- [1] N. Ferguson, B. Schneier, and T. Kohno, *Cryptography Engineering: Design Principles and Practical Applications*. 2010. doi: [10.1002/9781118722367](https://doi.org/10.1002/9781118722367).
- [2] M. Dworkin, “Recommendation for Block Cipher Modes of Operation,” 2001. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf>
- [3] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” 2007. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf>
- [4] R. L. Rivest, A. Shamir, and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [5] E. Barker and L. Chen, “Recommendation for Pair-Wise Key Establishment Schemes Using Integer Factorization Cryptography,” 2019. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-56Br2>
- [6] N. Koblitz, “Elliptic Curve Cryptosystems,” *Mathematics of Computation*, vol. 48, no. 177, pp. 203–209, 1987.
- [7] N. Gura, A. Patel, A. Wander, H. Eberle, and S. C. Shantz, “Comparing Elliptic Curve Cryptography and RSA on 8-bit CPUs,” in *Proceedings of the 6th International Workshop on Cryptographic Hardware and Embedded Systems (CHES)*, Springer, 2004, pp. 119–132. doi: [10.1007/978-3-540-28632-5_9](https://doi.org/10.1007/978-3-540-28632-5_9).